

Giáo trình Trí tuệ Nhân tạo & Ứng dụng

Hướng dẫn học tập trọng tâm chuẩn đại học

Đối tượng người học: sinh viên

Sách học tập dành cho sinh viên, bám sát nội dung tài liệu gốc.

Thời lượng ước tính: 15-20 giờ học

Chương 1: Chapter 1 giới thiệu về python cho trí

Nguyên lý và cơ chế hoạt động của Chapter 1 giới thiệu về python cho trí

Chương 2: Chapter 2 học có giám sát supervised learning

Nguyên lý và cơ chế hoạt động của Chapter 2 học có giám sát supervised learning

Chương 3: 5 phân cụm phân cấp hierarchical

Nguyên lý và cơ chế hoạt động của 27 3 5 phân cụm phân cấp hierarchical

Chương 4: Chapter 1 giới thiệu về python cho trí

Nguyên lý và cơ chế hoạt động của Chapter 1 giới thiệu về python cho trí

Chapter 1 giới thiệu về python cho trí

Chapter 1 Giới thiệu về Python cho Trí tuệ Nhân tạo Chương này giới thiệu các kiến thức lập trình cơ bản cần thiết cho các dự án Trí tuệ Nhân tạo (AI), sử dụng

Kết quả học tập chương:

Nắm vững và vận dụng Chapter 1 giới thiệu về python cho trí trong thực tiễn.

Nguyên lý và cơ chế hoạt động của Chapter 1 giới thiệu về python cho trí

Thời lượng: 20-30 phút

Ý tưởng cốt lõi (Core Idea):

Chương 1 được thiết kế để trang bị cho sinh viên có nền tảng C++ những kiến thức Python cơ bản và hệ sinh thái thư viện cần thiết cho Trí tuệ Nhân tạo, thông qua phương pháp học so sánh và các ví dụ thực tiễn.

Tại sao quan trọng (Why It Matters):

Việc hiểu rõ mục tiêu, cấu trúc và phương pháp tiếp cận của chương sẽ giúp sinh viên tối ưu hóa quá trình học tập, chuyển đổi kiến thức từ C++ sang Python một cách hiệu quả và nhanh chóng hòa nhập vào thế giới lập trình AI.

Mục tiêu bài học:

Hiểu được mục tiêu tổng thể và lý do Python được chọn làm ngôn ngữ chính cho AI trong chương này.

Nắm vững phương pháp giảng dạy so sánh giữa Python và C++ để tận dụng kiến thức nền tảng.

Nhận diện các nhóm kiến thức Python cơ bản và thư viện thiết yếu sẽ được giới thiệu.

Xác định được lộ trình học tập và các kỹ năng cần đạt được sau chương này.

Diễn giải kiến thức:

Chào mừng các em đến với Chương 1: Giới thiệu về Python cho Trí tuệ Nhân tạo. Chương này đóng vai trò là nền tảng quan trọng, giúp các em, những người đã có kinh nghiệm vững chắc với C++, chuyển đổi và làm quen với Python – ngôn ngữ lập trình chủ đạo trong lĩnh vực AI. ****Mục tiêu chính của chương**** là cung cấp các kiến thức lập trình Python cơ bản nhưng thiết yếu, cùng với việc giới thiệu các thư viện quan trọng, để các em có thể bắt đầu xây dựng và phát triển các dự án AI. Chúng ta sẽ không đi sâu vào từng chi tiết nhỏ của Python mà sẽ tập trung vào những khía cạnh trực tiếp phục vụ cho AI. ****Lý do chọn Python**** làm ngôn ngữ chính trong AI là vì sự đơn giản, tính linh hoạt cao và đặc biệt là hệ sinh thái thư viện vô cùng phong phú. Python cho phép các nhà nghiên cứu và phát triển AI tập trung vào thuật toán và mô hình thay vì phải lo lắng quá nhiều về cú pháp phức tạp hay quản lý bộ nhớ cấp thấp, điều mà C++ thường yêu cầu. Sự dễ đọc và dễ viết của Python cũng góp phần đẩy nhanh quá trình phát triển và thử nghiệm ý tưởng. ****Cơ chế hoạt động và phương pháp giảng dạy**** của chương này được thiết kế đặc biệt dành cho các em. Vì các em đã thành thạo C++, chúng ta sẽ áp dụng phương pháp học so sánh. Điều này có nghĩa là, khi giới thiệu một khái niệm mới trong Python (ví dụ: vòng lặp, hàm), chúng ta sẽ đồng thời nhắc lại cách các em đã thực hiện điều đó trong C++ và chỉ ra những điểm tương đồng cũng như khác biệt quan trọng. Cách tiếp cận này giúp các em dễ dàng liên hệ, hiểu sâu hơn về triết lý của Python và nhanh chóng nắm bắt các cú pháp mới mà không cảm thấy xa lạ. Các ví dụ mã lệnh Python sẽ được cung cấp kèm theo để minh họa rõ ràng. Trong chương này, chúng ta sẽ ôn lại các khái niệm Python thiết yếu như vòng lặp, cấu trúc điều khiển, và hàm. Sau đó, chúng ta sẽ dần chuyển sang các thư viện quan trọng như Num Py và Pandas, vốn là xương sống trong việc xử lý dữ liệu cho AI. Cuối cùng, các bài tập thực hành sẽ củng cố kiến thức và giúp các em tự tin hơn trong việc áp dụng Python vào các mô hình và dự án AI sau này.

Điểm trọng tâm cần nhớ:

Chương 1 tập trung giới thiệu Python cơ bản cho các dự án AI.

Python được chọn vì sự đơn giản, linh hoạt và hệ sinh thái thư viện phong phú trong AI.

Phương pháp giảng dạy chính là so sánh Python với C++ để tận dụng nền tảng kiến thức của sinh viên.

Các khái niệm Python cơ bản (vòng lặp, hàm) và thư viện thiết yếu (Num Py, Pandas) sẽ được giới thiệu.

Các thuật ngữ & khái niệm:

Python trong AI: Python là ngôn ngữ lập trình cấp cao, linh hoạt và dễ học, được sử dụng rộng rãi trong lĩnh vực Trí tuệ Nhân tạo nhờ cú pháp đơn giản, khả năng đọc hiểu cao và đặc biệt là hệ sinh thái thư viện mạnh mẽ (như Num Py, Pandas, Scikit-learn, Tensor Flow, Py Torch) hỗ trợ hiệu quả cho việc phát triển và triển khai các mô hình AI.

Học so sánh (Python vs C++): Một phương pháp sư phạm được áp dụng trong chương này, tận dụng kiến thức lập trình C++ sẵn có của sinh viên để giúp họ tiếp thu các khái niệm tương đương trong Python. Phương pháp này tập trung vào việc chỉ ra những điểm tương đồng và khác biệt về cú pháp, cấu trúc và triết lý lập trình giữa hai ngôn ngữ, giúp quá trình chuyển đổi và học tập trở nên hiệu quả và nhanh chóng hơn.

Ví dụ minh họa:

Khi giới thiệu về vòng lặp `for` trong Python, chương này sẽ không chỉ trình bày cú pháp `for i in range(N):` mà còn nhắc lại cách bạn đã sử dụng `for (int i = 0; i < N; ++i)` trong C++. Điều này giúp bạn dễ dàng nhận ra rằng `range(N)` trong Python tương đương với điều kiện `i < N` trong C++, tạo ra một dãy số từ 0 đến N-1, và cách Python xử lý việc lặp qua các phần tử một cách tự nhiên hơn.

Sai lầm phổ biến & Cách hiểu đúng:

Hiểu sai: Sinh viên có nền tảng C++ thường nghĩ rằng Python chỉ là một ngôn ngữ 'đơn giản hơn' và có thể bỏ qua việc học kỹ các khái niệm cơ bản, chỉ cần 'đọc qua' là đủ.

Đúng là: Mặc dù Python có cú pháp đơn giản hơn C++, nhưng nó có triết lý thiết kế và cách hoạt động khác biệt đáng kể, đặc biệt là trong quản lý bộ nhớ, kiểu dữ liệu động và cách xử lý các cấu trúc dữ liệu. Việc không nắm vững các khái niệm cơ bản của Python có thể dẫn đến các lỗi logic khó phát hiện, mã kém hiệu quả hoặc không tận dụng được hết sức mạnh của các thư viện AI. Do đó, việc học kỹ lưỡng từ đầu là rất quan trọng.

Hoạt động thực hành:

Hãy dành 5 phút để đọc lướt qua mục lục của Chương 1 và các chương tiếp theo trong giáo trình. Sau đó, tự đặt câu hỏi và suy nghĩ về cách các kiến thức Python cơ bản trong chương này sẽ là nền tảng cho các chủ đề AI phức tạp hơn ở các chương sau (ví dụ: học máy có giám sát, mạng nơ-ron).

Kiểm tra nhanh (Quick Check):

Q: Mục tiêu chính của Chương 1 là gì?

A: Giới thiệu các kiến thức lập trình Python cơ bản cần thiết cho các dự án Trí tuệ Nhân tạo (AI). (Chương này được thiết kế để trang bị nền tảng Python cho sinh viên để họ có thể bắt đầu làm việc với AI, không phải là một khóa học Python tổng quát.)

Q: Chương này được thiết kế đặc biệt cho đối tượng sinh viên nào?

A: Sinh viên đã có kiến thức vững chắc về lập trình C++. (Giáo trình tận dụng nền tảng C++ của sinh viên để áp dụng phương pháp học so sánh, giúp quá trình chuyển đổi ngôn ngữ hiệu quả hơn.)

Q: Tại sao Python lại được ưa chuộng trong lĩnh vực AI?

A: Python đơn giản, linh hoạt, dễ đọc, dễ viết và có hệ sinh thái thư viện phong phú hỗ trợ mạnh mẽ cho AI. (Những đặc điểm này giúp các nhà phát triển AI tập trung vào giải quyết vấn đề thay vì các chi tiết kỹ thuật phức tạp của ngôn ngữ.)

Tóm tắt chương:

Chương 1 đã giới thiệu tổng quan về vai trò của Python trong Trí tuệ Nhân tạo và cách chương này sẽ trang bị kiến thức cho sinh viên có nền tảng C++. Chúng ta đã tìm hiểu về mục tiêu, phương pháp học so sánh Python và C++, cũng như các khái niệm Python cơ bản và thư viện thiết yếu sẽ được đề cập. Chương này đặt nền móng vững chắc để sinh viên tự tin tiếp cận các chủ đề AI phức tạp hơn trong các chương tiếp theo.

Chapter 2 học có giám sát supervised learning

Chapter 2 Học có giám sát – Supervised Learning 2.1 Giới thiệu Học có giám sát (Supervised Learning) là một nhánh quan trọng trong học máy (Machine Learning).

Kết quả học tập chương:

Nắm vững và vận dụng Chapter 2 học có giám sát supervised learning trong thực tiễn.

Nguyên lý và cơ chế hoạt động của Chapter 2 học có giám sát supervised learning

Thời lượng: 20-30 phút

Ý tưởng cốt lõi (Core Idea):

Học có giám sát là một phương pháp trong học máy, nơi mô hình học từ dữ liệu đã có nhãn (đầu vào và đầu ra đã biết) để dự đoán đầu ra cho dữ liệu mới. Nó bao gồm hai loại bài toán chính: Hồi quy (Regression) cho giá trị liên tục và Phân loại (Classification) cho nhãn rời rạc.

Tại sao quan trọng (Why It Matters):

Học có giám sát là nền tảng cho vô số ứng dụng trí tuệ nhân tạo trong đời sống và công nghiệp, từ dự đoán giá nhà, nhận diện hình ảnh, phân loại email spam cho đến chẩn đoán bệnh, giúp tự động hóa và đưa ra quyết định thông minh hơn.

Mục tiêu bài học:

Hiểu định nghĩa và vai trò của Học có giám sát trong lĩnh vực học máy.

Nắm vững ý tưởng cơ bản và cơ chế hoạt động của các mô hình Học có giám sát.

Phân biệt được hai loại bài toán chính trong Học có giám sát: Hồi quy và Phân loại, cùng với các ví dụ minh họa.

Diễn giải kiến thức:

Học có giám sát (Supervised Learning) là một trong những nhánh quan trọng nhất của học máy (Machine Learning). Ý tưởng cốt lõi của phương pháp này là học một mô hình từ một tập dữ liệu đã có sẵn các cặp 'đầu vào' (features) và 'đầu ra' (labels) tương ứng. Nói cách khác, chúng ta cung cấp cho mô hình các ví dụ đã được 'giám sát' hay 'gắn nhãn' đúng, để nó có thể học cách ánh xạ từ đầu vào đến đầu ra. Cơ chế hoạt động diễn ra như sau: Đầu tiên, chúng ta có một tập dữ liệu huấn luyện bao gồm nhiều mẫu, mỗi mẫu có các đặc trưng đầu vào và một nhãn đầu ra đã biết. Mô hình sẽ phân tích mối quan hệ giữa các đặc trưng đầu vào và nhãn đầu ra này. Mục tiêu là 'học' một hàm hoặc một quy tắc tổng quát từ dữ liệu này. Sau khi quá trình huấn luyện hoàn tất, mô hình đã học được có thể được sử dụng để dự đoán đầu ra cho các dữ liệu đầu vào mới, chưa từng thấy trước đây mà không cần biết trước nhãn của chúng. Học có giám sát được chia thành hai loại bài toán chính dựa trên tính chất của đầu ra cần dự đoán: 1.

****Hồi quy (Regression)**:** Đây là loại bài toán mà đầu ra cần dự đoán là một giá trị số liên tục. Ví dụ, dự đoán giá nhà dựa trên diện tích, số phòng, vị trí; dự đoán nhiệt độ ngày mai; hoặc dự đoán doanh số bán hàng của một sản phẩm. Kết quả dự đoán không phải là một danh mục cụ thể mà là một con số nằm trong một khoảng giá trị. 2. ****Phân loại (Classification)**:** Đây là loại bài toán mà đầu ra cần dự đoán là một nhãn hoặc danh mục rời rạc. Ví dụ, phân loại email là 'spam' hay 'không spam'; nhận diện hình ảnh là 'mèo' hay 'chó'; hoặc chẩn đoán bệnh là 'có bệnh' hay 'không bệnh'. Kết quả dự đoán là việc gán một mẫu dữ liệu vào một trong các lớp (class) đã định trước.

Điểm trọng tâm cần nhớ:

Học có giám sát yêu cầu dữ liệu huấn luyện phải có nhãn (đầu vào và đầu ra đã biết).

Mục tiêu chính là xây dựng một mô hình có khả năng dự đoán đầu ra cho dữ liệu mới.

Hai loại bài toán cơ bản là Hồi quy (dự đoán giá trị số liên tục) và Phân loại (dự đoán nhãn rời rạc).

Các thuật ngữ & khái niệm:

Học có giám sát (Supervised Learning): Một phương pháp trong học máy, nơi mô hình được huấn luyện trên một tập dữ liệu đã có sẵn các cặp đầu vào (features) và đầu ra mong muốn (labels). Mục tiêu là học một ánh xạ từ đầu vào đến đầu ra để có thể dự đoán đầu ra cho dữ liệu mới chưa từng thấy.

Đặc trưng (Features): Các thuộc tính hoặc biến đầu vào của dữ liệu được sử dụng để huấn luyện mô hình và đưa ra dự đoán. Ví dụ: diện tích nhà, số phòng, màu sắc.

Nhãn (Labels): Giá trị đầu ra mong muốn hoặc kết quả mục tiêu tương ứng với mỗi tập hợp đặc trưng đầu vào trong dữ liệu huấn luyện. Ví dụ: giá nhà, 'spam'/'không spam'.

Hồi quy (Regression): Một loại bài toán trong học có giám sát nhằm dự đoán một giá trị đầu ra liên tục, ví dụ như giá nhà, nhiệt độ, doanh số bán hàng.

Phân loại (Classification): Một loại bài toán trong học có giám sát nhằm dự đoán một nhãn hoặc danh mục rời rạc, ví dụ như phân loại email là spam hay không spam, nhận diện hình ảnh là mèo hay chó.

Ví dụ minh họa:

Để minh họa, hãy xem xét hai ví dụ: 1. ****Ví dụ Hồi quy****: Bạn muốn dự đoán giá bán của một căn hộ mới trên thị trường. Dữ liệu huấn luyện của bạn sẽ bao gồm thông tin về các căn hộ đã bán trước đó (diện tích, số phòng ngủ, vị trí, năm xây dựng – đây là các 'features') và giá bán thực tế của chúng (đây là 'labels'). Mô hình học có giám sát sẽ học mối quan hệ giữa các đặc trưng này và giá bán để khi bạn nhập thông tin về một căn hộ mới, nó có thể dự đoán giá bán ước tính. 2. ****Ví dụ Phân loại****: Bạn muốn xây dựng một hệ thống để tự động phân loại email đến là 'spam' hay 'không spam'. Dữ liệu huấn luyện của bạn sẽ bao gồm hàng ngàn email (nội dung, người gửi, tiêu đề – đây là các 'features') và nhãn đã được gán thủ công cho từng email là 'spam' hoặc 'không spam' (đây là 'labels'). Mô hình sẽ học các mẫu đặc trưng của email spam và email hợp lệ để khi một email mới đến, nó có thể phân loại chính xác.

Sai lầm phổ biến & Cách hiểu đúng:

Hiểu sai: Sinh viên thường nhầm lẫn giữa Học có giám sát (Supervised Learning) và Học không giám sát (Unsupervised Learning).

Đúng là: Điểm khác biệt cốt lõi là Học có giám sát luôn yêu cầu dữ liệu huấn luyện phải có 'nhãn' (labels) – tức là chúng ta biết trước đầu ra đúng cho mỗi đầu vào. Ngược lại, Học không giám sát làm việc với dữ liệu không có nhãn, mục tiêu là tìm kiếm cấu trúc ẩn, mẫu hoặc nhóm trong dữ liệu mà không có bất kỳ thông tin đầu ra nào được cung cấp trước.

Hoạt động thực hành:

Hãy tìm và mô tả 3 ví dụ thực tế khác nhau cho bài toán Hồi quy và 3 ví dụ cho bài toán Phân loại. Đối với mỗi ví dụ, hãy giải thích tại sao nó thuộc loại đó và xác định rõ ràng đâu là 'đặc trưng đầu vào' (features) và đâu là 'nhãn đầu ra' (labels) mà mô hình sẽ học.

Kiểm tra nhanh (Quick Check):

Q: Điểm khác biệt cơ bản nhất giữa Học có giám sát và Học không giám sát là gì?

A: Học có giám sát yêu cầu dữ liệu có nhãn (đầu vào và đầu ra đã biết), trong khi Học không giám sát làm việc với dữ liệu không có nhãn. (Sự hiện diện của 'nhãn' (labels) trong dữ liệu huấn luyện là yếu tố phân biệt chính. Trong học có giám sát, mô hình được 'dạy' bằng các ví dụ đúng, còn trong học không giám sát, mô hình tự tìm kiếm cấu trúc mà không có 'giáo viên'.)

Q: Nếu bạn muốn dự đoán nhiệt độ cao nhất của ngày mai ở Hà Nội, đây là bài toán Hồi quy hay Phân loại?

A: Hồi quy. (Nhiệt độ là một giá trị số liên tục (ví dụ: 28.5 độ C, 30 độ C), không phải là một danh mục hay nhãn rời rạc. Do đó, đây là một bài toán Hồi quy.)

Q: Để phân loại một bức ảnh là 'mèo', 'chó' hay 'chim', bạn cần loại dữ liệu đầu ra nào?

A: Nhãn rời rạc (ví dụ: 'mèo', 'chó', 'chim'). (Đây là một bài toán phân loại đa lớp, nơi đầu ra thuộc về một trong các danh mục đã định trước. Mỗi bức ảnh sẽ được gán một nhãn cụ thể trong số các lựa chọn.)

Tóm tắt chương:

Chương 2 giới thiệu về Học có giám sát (Supervised Learning), một nhánh quan trọng của học máy. Chúng ta đã tìm hiểu nguyên lý hoạt động cơ bản của nó, dựa trên việc học từ dữ liệu có nhãn (đầu vào và đầu ra đã biết) để xây dựng mô hình dự đoán. Chương này cũng làm rõ hai loại bài toán chính trong học có giám sát: Hồi quy, dùng để dự đoán các giá trị số liên tục, và Phân loại, dùng để dự đoán các nhãn hoặc danh mục rời rạc, cùng với các ví dụ minh họa cụ thể cho từng loại.

5 phân cụm phân cấp hierarchical

27 3.5 Phân Cụm Phân Cấp (Hierarchical Clustering) 28 3.5.1 Giới thiệu 28
3.5.2 Cách Thuật Toán Hoạt Động 28 3.5.3 Ví dụ Minh Họa 29 3.5.4 Ứng Dụng Thực
Tiễn 30

Kết quả học tập chương:

Nắm vững và vận dụng 27 3 5 phân cụm phân cấp hierarchical trong thực tiễn.

Nguyên lý và cơ chế hoạt động của 27 3 5 phân cụm phân cấp hierarchical

Thời lượng: 20-30 phút

Ý tưởng cốt lõi (Core Idea):

Phân cụm phân cấp là một phương pháp phân cụm dữ liệu xây dựng một cấu trúc phân cấp các cụm, không yêu cầu xác định trước số lượng cụm K.

Tại sao quan trọng (Why It Matters):

Phương pháp này giúp khám phá cấu trúc tự nhiên của dữ liệu, cung cấp cái nhìn sâu sắc về mối quan hệ giữa các điểm dữ liệu và cho phép người dùng linh hoạt chọn số lượng cụm phù hợp sau khi quá trình phân cụm hoàn tất.

Mục tiêu bài học:

Giải thích được khái niệm và mục tiêu của phân cụm phân cấp.

Mô tả được hai phương pháp chính: gom cụm (Agglomerative) và chia tách (Divisive).

Hiểu và áp dụng các tiêu chí liên kết (linkage criteria) khác nhau.

Đọc và diễn giải được biểu đồ cây (Dendrogram).

Diễn giải kiến thức:

Phân cụm phân cấp (Hierarchical Clustering) là một kỹ thuật phân cụm dữ liệu mạnh mẽ, tạo ra một cấu trúc phân cấp các cụm, thường được biểu diễn dưới dạng biểu đồ cây (dendrogram). Điểm đặc biệt của phương pháp này là nó không yêu cầu người dùng phải xác định trước số lượng cụm K, điều này khác biệt so với các thuật toán như K-Means. Có hai cách tiếp cận chính trong phân cụm phân cấp: 1. **Phân cụm gom cụm (Agglomerative Hierarchical Clustering - 'bottom-up')**: Đây là phương pháp phổ biến hơn. Nó bắt đầu bằng cách coi mỗi điểm dữ liệu là một cụm riêng biệt. Sau đó, thuật toán lặp đi lặp lại việc hợp nhất hai cụm gần nhất thành một cụm lớn hơn. Quá trình này tiếp tục cho đến khi tất cả các điểm dữ liệu được hợp nhất thành một cụm duy nhất, hoặc một tiêu chí dừng nào đó được thỏa mãn. 2. **Phân cụm chia tách (Divisive Hierarchical Clustering - 'top-down')**: Phương pháp này bắt đầu với tất cả các điểm dữ liệu nằm trong một cụm lớn duy nhất. Sau đó, thuật toán lặp đi lặp lại việc chia tách cụm lớn nhất thành hai cụm nhỏ hơn. Quá trình này tiếp tục cho đến khi mỗi điểm dữ liệu trở thành một cụm riêng biệt, hoặc một tiêu chí dừng nào đó được thỏa mãn. Để xác định 'gần nhất' giữa các cụm, thuật toán sử dụng các **tiêu chí liên kết (linkage criteria)** và **đo lường khoảng cách (distance metrics)**. Các tiêu chí liên kết phổ biến bao gồm: **Liên kết đơn (Single Linkage)**: Khoảng cách giữa hai cụm được định nghĩa là khoảng cách nhỏ nhất giữa bất kỳ cặp điểm nào, trong đó một điểm thuộc cụm này và một điểm thuộc cụm kia. Phương pháp này có xu hướng tạo ra các cụm 'dài' và 'mỏng'. **Liên kết hoàn chỉnh (Complete Linkage)**: Khoảng cách giữa hai cụm được định nghĩa là khoảng cách lớn nhất giữa bất kỳ cặp điểm nào, trong đó một điểm thuộc cụm này và một điểm thuộc cụm kia. Phương pháp này có xu hướng tạo ra các cụm 'tròn' và nhỏ gọn hơn. **Liên kết trung bình (Average Linkage)**: Khoảng cách giữa hai cụm được định nghĩa là khoảng cách trung bình của tất cả các cặp điểm có thể có giữa hai cụm. **Phương pháp Ward (Ward's Method)**: Mục tiêu là giảm thiểu phương sai tổng thể trong các cụm được hợp nhất. Nó tìm cặp cụm mà việc hợp nhất chúng sẽ dẫn đến sự gia tăng nhỏ nhất về tổng bình phương khoảng cách từ các điểm đến tâm cụm của chúng (tức là giảm thiểu sự mất mát thông tin). Kết quả của phân cụm phân cấp thường được trực quan hóa bằng **biểu đồ cây (Dendrogram)**. Biểu đồ này hiển thị trình tự các cụm được hợp nhất (hoặc chia tách) và khoảng cách tương ứng tại mỗi bước. Bằng cách cắt ngang dendrogram ở một mức độ khoảng cách nhất định, chúng ta có thể xác định số lượng cụm mong muốn.

Điểm trọng tâm cần nhớ:

Phân cụm phân cấp xây dựng một cấu trúc cây (phân cấp) của các cụm.

Không cần xác định số lượng cụm K trước.

Hai phương pháp chính là Agglomerative (gom cụm từ dưới lên) và Divisive (chia tách từ trên xuống).

Tiêu chí liên kết (Single, Complete, Average Linkage, Ward's Method) và đo lường khoảng cách là yếu tố then chốt để xác định sự 'gần' giữa các cụm.

Biểu đồ cây (Dendrogram) là công cụ trực quan hóa chính để hiểu kết quả và chọn số lượng cụm.

Các thuật ngữ & khái niệm:

Phân cụm phân cấp (Hierarchical Clustering): Một phương pháp phân cụm dữ liệu xây dựng một cấu trúc phân cấp các cụm, thường được biểu diễn dưới dạng biểu đồ cây (dendrogram), không yêu cầu xác định trước số lượng cụm.

Phân cụm gom cụm (Agglomerative Clustering): Phương pháp 'từ dưới lên' bắt đầu với mỗi điểm dữ liệu là một cụm riêng biệt và liên tục hợp nhất các cụm gần nhất cho đến khi đạt được một cụm duy nhất hoặc tiêu chí dừng.

Phân cụm chia tách (Divisive Clustering): Phương pháp 'từ trên xuống' bắt đầu với tất cả các điểm dữ liệu trong một cụm duy nhất và liên tục chia tách cụm lớn nhất thành các cụm nhỏ hơn cho đến khi mỗi điểm là một cụm riêng biệt hoặc tiêu chí dừng.

Biểu đồ cây (Dendrogram): Một biểu đồ dạng cây trực quan hóa cấu trúc phân cấp của các cụm, hiển thị trình tự hợp nhất/chia tách và khoảng cách tương ứng giữa các cụm.

Tiêu chí liên kết (Linkage Criteria): Quy tắc được sử dụng để tính khoảng cách giữa hai cụm, ví dụ: liên kết đơn, liên kết hoàn chỉnh, liên kết trung bình, phương pháp Ward.

Khoảng cách Euclidean: Một phép đo khoảng cách phổ biến giữa hai điểm trong không gian Euclidean, thường được sử dụng trong phân cụm để xác định sự 'gần' của các điểm dữ liệu.

Ví dụ minh họa:

Giả sử chúng ta có 5 điểm dữ liệu 1D: A(1), B(2), C(5), D(6), E(7). **Các bước của Agglomerative Clustering (sử dụng khoảng cách Euclidean và liên kết đơn):**

1. **Khởi tạo:** Mỗi điểm là một cụm riêng biệt: {A}, {B}, {C}, {D}, {E}.
2. **Bước 1:** Tìm hai cụm gần nhất. Khoảng cách(A,B) = $|1-2| = 1$. Hợp nhất {A} và {B} thành {A,B}. Các cụm: {A,B}, {C}, {D}, {E}.
3. **Bước 2:** Tìm hai cụm gần nhất. Khoảng cách(D,E) = $|6-7| = 1$. Hợp nhất {D} và {E} thành {D,E}. Các cụm: {A,B}, {C}, {D,E}.
4. **Bước 3:** Tìm hai cụm gần nhất. Khoảng cách(C,D) = $|5-6| = 1$. Khoảng cách(C, {D,E}) = $\min(|5-6|, |5-7|) = \min(1, 2) = 1$. Hợp nhất {C} và {D,E} thành {C,D,E}. Các cụm: {A,B}, {C,D,E}.
5. **Bước 4:** Tìm hai cụm gần nhất. Khoảng cách({A,B}, {C,D,E}) = $\min(|1-5|, |1-6|, |1-7|, |2-5|, |2-6|, |2-7|) = \min(4, 5, 6, 3, 4, 5) = 3$. Hợp nhất {A,B} và {C,D,E} thành {A,B,C,D,E}. Các cụm: {A,B,C,D,E}.

Biểu đồ cây (dendrogram) sẽ thể hiện rõ trình tự hợp nhất này, cho phép chúng ta 'cắt' ở các mức khoảng cách khác nhau để có được số lượng cụm mong muốn (ví dụ, cắt ở khoảng cách 1 sẽ cho 3 cụm: {A,B}, {C}, {D,E}).

Sai lầm phổ biến & Cách hiểu đúng:

Hiểu sai: Nhầm lẫn rằng phân cụm phân cấp tự động cho ra số lượng cụm tối ưu.

Đúng là: Phân cụm phân cấp tạo ra một cấu trúc phân cấp. Việc chọn số lượng cụm K cuối cùng vẫn là một quyết định của người dùng, thường dựa trên việc phân tích biểu đồ cây (dendrogram) và kiến thức chuyên môn về miền dữ liệu. Dendrogram giúp trực quan hóa các cụm tiềm năng ở các mức độ phân cấp khác nhau, nhưng không tự động chọn K.

Hoạt động thực hành:

Vẽ biểu đồ cây (dendrogram) cho một tập dữ liệu nhỏ gồm 6 điểm 1D: {2, 3, 8, 9, 10, 15} sử dụng phương pháp Agglomerative Clustering với khoảng cách Euclidean và tiêu chí liên kết hoàn chỉnh (Complete Linkage). Xác định số lượng cụm nếu bạn cắt dendrogram ở mức khoảng cách 2.

Kiểm tra nhanh (Quick Check):

Q: Điểm khác biệt chính giữa Agglomerative và Divisive Hierarchical Clustering là gì?

A: Agglomerative bắt đầu với các cụm đơn lẻ và hợp nhất chúng (bottom-up), trong khi Divisive bắt đầu với một cụm lớn và chia tách nó (top-down). (Agglomerative là phương pháp gom cụm từ dưới lên, từng bước hợp nhất các cụm nhỏ nhất. Divisive là phương pháp chia tách từ trên xuống, bắt đầu từ một cụm lớn và chia nhỏ dần.)

Q: Nếu bạn muốn các cụm có hình dạng 'tròn' và nhỏ gọn, bạn nên ưu tiên tiêu chí liên kết nào?

A: Liên kết hoàn chỉnh (Complete Linkage) hoặc Phương pháp Ward (Ward's Method). (Liên kết hoàn chỉnh có xu hướng tạo ra các cụm nhỏ gọn hơn vì nó dựa trên khoảng cách lớn nhất giữa các điểm trong hai cụm. Phương pháp Ward cũng hướng tới việc tạo ra các cụm có phương sai nội bộ thấp, tức là các cụm chặt chẽ.)

Q: Biểu đồ cây (Dendrogram) có vai trò gì trong phân cụm phân cấp?

A: Biểu đồ cây trực quan hóa cấu trúc phân cấp của các cụm và giúp người dùng quyết định số lượng cụm K bằng cách 'cắt' biểu đồ ở một mức khoảng cách nhất định. (Dendrogram là công cụ thiết yếu để hiểu quá trình hợp nhất/chia tách và là cơ sở để đưa ra quyết định về số lượng cụm cuối cùng dựa trên ngưỡng khoảng cách.)

Tóm tắt chương:

Chương 3.5 giới thiệu về Phân Cụm Phân Cấp (Hierarchical Clustering), một phương pháp phân cụm không yêu cầu xác định trước số lượng cụm. Chương này đã trình bày nguyên lý hoạt động của hai phương pháp chính là gom cụm (Agglomerative) và chia tách (Divisive), cùng với các tiêu chí liên kết quan trọng như Single, Complete, Average Linkage và Ward's Method. Sinh viên cũng đã được học cách trực quan hóa kết quả bằng biểu đồ cây (Dendrogram) và hiểu cách chọn số lượng cụm phù hợp từ biểu đồ này.

Chapter 1 giới thiệu về python cho trí

Chapter 1 Giới thiệu về Python cho Trí tuệ Nhân tạo print(i) Đoạn mã trên sẽ in ra các số từ 0 đến 4.

Kết quả học tập chương:

Nắm vững và vận dụng Chapter 1 giới thiệu về python cho trí trong thực tiễn.

Nguyên lý và cơ chế hoạt động của Chapter 1 giới thiệu về python cho trí

Thời lượng: 20-30 phút

Ý tưởng cốt lõi (Core Idea):

Chương này đặt nền móng vững chắc cho sinh viên về lập trình Python trong bối cảnh Trí tuệ Nhân tạo, tập trung vào các khái niệm cơ bản và so sánh với C++ để tận dụng kiến thức nền tảng sẵn có của người học.

Tại sao quan trọng (Why It Matters):

Python là ngôn ngữ lập trình chủ đạo trong lĩnh vực AI nhờ sự đơn giản, linh hoạt và hệ sinh thái thư viện phong phú. Việc nắm vững các nguyên lý và cấu trúc cơ bản của Python ngay từ đầu là yếu tố then chốt để phát triển các mô hình và dự án AI hiệu quả.

Mục tiêu bài học:

Hiểu được vai trò và tầm quan trọng của Python trong lĩnh vực Trí tuệ Nhân tạo.

Nắm vững các khái niệm lập trình Python cơ bản như vòng lặp, cấu trúc điều khiển và hàm.

Phân biệt được sự khác biệt cơ bản giữa cú pháp Python và C++ đối với các cấu trúc lập trình tương đương.

Nhận diện được các thư viện Python thiết yếu cho xử lý dữ liệu trong AI (NumPy, Pandas).

Diễn giải kiến thức:

Chương 1, với tiêu đề 'Giới thiệu về Python cho Trí tuệ Nhân tạo', được thiết kế để cung cấp cho sinh viên một cái nhìn tổng quan và nền tảng vững chắc về việc sử dụng Python trong các dự án AI. Nguyên lý cốt lõi của chương là xây dựng kiến thức từ những gì sinh viên đã biết (C++) để dễ dàng tiếp cận một ngôn ngữ mới (Python).

Cơ chế hoạt động của chương:

- Giới thiệu tổng quan về Python và AI:** Chương bắt đầu bằng việc khẳng định Python là ngôn ngữ chính cho AI, nhấn mạnh sự đơn giản, linh hoạt và hệ sinh thái thư viện phong phú của nó. Điều này giúp sinh viên hình dung được bức tranh lớn và động lực học tập.
- Ôn tập và so sánh các khái niệm cơ bản:** Thay vì dạy lại từ đầu, chương tập trung vào việc ôn lại các khái niệm lập trình thiết yếu của Python như vòng lặp (ví dụ: `for` loop), cấu trúc điều khiển (ví dụ: `if/else`) và hàm (`def`). Đặc biệt, chương này tận dụng kiến thức C++ của sinh viên bằng cách so sánh trực tiếp cú pháp và cách hoạt động giữa Python và C++ (ví dụ: hàm `range()` trong Python so với điều kiện `i < N` trong C++). Điều này giúp sinh viên dễ dàng chuyển đổi tư duy và cú pháp.
- Giới thiệu hàm trong Python:** Hàm là một khối xây dựng cơ bản trong mọi ngôn ngữ lập trình. Chương giải thích cách định nghĩa hàm bằng từ khóa `def` và cách sử dụng `return` để trả về giá trị, kèm theo ví dụ minh họa cụ thể.
- Nhấn mạnh tầm quan trọng của thư viện:** Chương giới thiệu sơ lược về các thư viện Python quan trọng như Num Py và Pandas, vốn là nền tảng cho việc xử lý dữ liệu trong AI. Mặc dù chưa đi sâu vào chi tiết, việc giới thiệu sớm giúp sinh viên nhận thức được vai trò của các công cụ này. Thông qua cấu trúc này, chương không chỉ truyền đạt kiến thức Python mà còn giúp sinh viên phát triển tư duy lập trình linh hoạt, sẵn sàng cho các bài toán AI phức tạp hơn.

Điểm trọng tâm cần nhớ:

Python là ngôn ngữ chính cho AI nhờ sự đơn giản và hệ sinh thái thư viện mạnh mẽ.

Các khái niệm cơ bản của Python bao gồm vòng lặp (`for`, `while`), cấu trúc điều khiển (`if`, `elif`, `else`) và hàm (`def`).

Hàm `range(N)` trong Python tạo ra dãy số từ 0 đến N-1, khác với điều kiện `i < N` trong C++.

Hàm trong Python được định nghĩa bằng từ khóa `def` và sử dụng `return` để trả về giá trị.

Num Py và Pandas là hai thư viện Python quan trọng cho xử lý dữ liệu trong AI.

Các thuật ngữ & khái niệm:

Chapter 1 giới thiệu về python cho trí: Chương học đầu tiên trong giáo trình, tập trung giới thiệu các kiến thức lập trình Python cơ bản và thiết yếu cho các dự án Trí tuệ Nhân tạo, đặc biệt dành cho sinh viên đã có nền tảng C++.

Python cho Trí tuệ Nhân tạo: Việc sử dụng ngôn ngữ lập trình Python làm công cụ chính để phát triển các ứng dụng, mô hình và thuật toán trong lĩnh vực Trí tuệ Nhân tạo, tận dụng sự đơn giản, linh hoạt và hệ sinh thái thư viện phong phú của nó.

Vòng lặp và Cấu trúc điều khiển (Python): Các cấu trúc lập trình cơ bản trong Python cho phép thực thi một khối mã nhiều lần (vòng lặp `for`, `while`) hoặc thực thi có điều kiện (`if`, `elif`, `else`), là nền tảng cho mọi thuật toán.

Hàm trong Python: Một khối mã được tổ chức để thực hiện một nhiệm vụ cụ thể, được định nghĩa bằng từ khóa `def` và có thể nhận tham số cũng như trả về giá trị bằng từ khóa `return`.

Ví dụ minh họa:

Để minh họa sự khác biệt giữa Python và C++ trong vòng lặp và cách định nghĩa hàm trong Python, chúng ta xem xét các ví dụ sau: ****1. Vòng lặp `for` và hàm `range()`:**** Trong C++, để in các số từ 0 đến 4, bạn thường viết: `cpp for (int i = 0; i < 5; ++i) { cout << i << endl; }` Trong Python, đoạn mã tương đương sẽ là: `python for i in range(5): print(i)` Ở đây, `range(5)` tạo ra một dãy số từ 0 đến 4 (tức là N-1), đơn giản và trực quan hơn. ****2. Định nghĩa và gọi hàm:**** Trong C++, một hàm cộng hai số có thể trông như sau: `cpp int add_numbers(int a, int b) { return a + b; }` // Gọi hàm `int result = add_numbers(3, 4); cout << result << endl; // In ra 7` Trong Python, hàm tương tự được định nghĩa bằng từ khóa `def`: `python # Function that adds two numbers def add_numbers(a, b): return a + b # Function call result = add_numbers(3, 4) print(result) # In ra 7` Ví dụ này cho thấy cú pháp Python gọn gàng và dễ đọc hơn, không yêu cầu khai báo kiểu dữ liệu cho tham số hay giá trị trả về.

Sai lầm phổ biến & Cách hiểu đúng:

Hiểu sai: Sinh viên từ C++ thường nhầm lẫn về cách hoạt động của hàm `range(N)` trong Python, cho rằng nó sẽ bao gồm cả số N, giống như điều kiện `i <= N` trong C++.

Đúng là: Hàm `range(N)` trong Python tạo ra một dãy số nguyên bắt đầu từ 0 và kết thúc ở N-1. Nó không bao gồm giá trị N. Điều này là một quy ước phổ biến trong Python và cần được ghi nhớ để tránh lỗi ngoài phạm vi (off-by-one errors).

Hoạt động thực hành:

1. ****Bài tập vòng lặp:**** Viết một chương trình Python sử dụng vòng lặp `for` để in ra tất cả các số chẵn từ 10 đến 50 (bao gồm cả 10 và 50). 2. ****Bài tập hàm:**** Viết một hàm Python có tên `reverse\_string` nhận vào một chuỗi (string) làm tham số và trả về chuỗi đó đã được đảo ngược. Ví dụ: `reverse\_string("hello")` sẽ trả về `"olleh"`.

Kiểm tra nhanh (Quick Check):

Q: Tại sao Python lại được lựa chọn là ngôn ngữ chính cho các dự án Trí tuệ Nhân tạo?

A: Python được lựa chọn vì sự đơn giản, dễ học, cú pháp rõ ràng, tính linh hoạt cao và đặc biệt là có một hệ sinh thái thư viện phong phú (như Num Py, Pandas, Scikit-learn, Tensor Flow, Py Torch) hỗ trợ mạnh mẽ cho các tác vụ AI và Machine Learning. (Các yếu tố này giúp các nhà phát triển AI tập trung vào logic thuật toán hơn là các chi tiết phức tạp của ngôn ngữ, tăng tốc độ phát triển và thử nghiệm.)

Q: Hàm `range(10)` trong Python sẽ tạo ra dãy số nào?

A: Hàm `range(10)` sẽ tạo ra dãy số nguyên từ 0 đến 9. (Trong Python, `range(N)` luôn tạo ra một dãy số bắt đầu từ 0 và kết thúc ở N-1. Đây là một điểm khác biệt quan trọng so với một số ngôn ngữ khác.)

Q: Để định nghĩa một hàm trong Python, từ khóa nào được sử dụng?

A: Từ khóa `def` được sử dụng để định nghĩa một hàm trong Python. (Ví dụ: `def my_function(param1, param2):` là cách khai báo một hàm cơ bản.)

Tóm tắt chương:

Chương 1 đã giới thiệu các kiến thức nền tảng về Python, đặc biệt nhấn mạnh vai trò của nó trong Trí tuệ Nhân tạo. Sinh viên đã được ôn lại các khái niệm lập trình cơ bản như vòng lặp, cấu trúc điều khiển và hàm, đồng thời được so sánh với C++ để dễ dàng chuyển đổi tư duy. Chương cũng giới thiệu sơ lược về các thư viện xử lý dữ liệu quan trọng như Num Py và Pandas, đặt nền móng vững chắc cho việc phát triển các dự án AI trong tương lai.

Thuật ngữ & Khái niệm (Glossary)

Chapter 1 giới thiệu về python cho trí (Chương 1): Chapter 1 Giới thiệu về Python cho Trí tuệ Nhân tạo Chương này giới thiệu các kiến thức lập trình cơ bản cần thiết cho các dự án Trí tuệ Nhân tạo (AI), sử dụng Python làm ngôn ngữ

Chapter 2 học có giám sát supervised learning (Chương 2): Chapter 2 Học có giám sát – Supervised Learning 2.1 Giới thiệu Học có giám sát (Supervised Learning) là một nhánh quan trọng trong học máy (Machine Learning).

3.5 phân cụm phân cấp hierarchical (Chương 3): 27 3.5 Phân Cụm Phân Cấp (Hierarchical Clustering) 28 3.5.1 Giới thiệu 28 3.5.2 Cách Thuật Toán Hoạt Động 28 3.5.3 Ví dụ Minh Họa 29 3.5.4 Ứng Dụng Thực Tiễn 30 4 Đánh Giá Trong Tr

Chapter 5 neural networks 5.1 kiến thức (Chương 6): Chapter 5 Neural Networks 5.1 Kiến thức cơ bản về Perceptron Lý thuyết Perceptron là dạng đơn giản nhất của mạng nơ-ron.

Chapter 8 computer vision 4 một số ứng dụng (Chương 6): Chapter 8 Computer Vision 4.

3 lan truyền ngược backpropagation tóm tắt (Chương 6): 5.3 Lan truyền ngược (Backpropagation) Tóm tắt • Cho phép huấn luyện mạng nơ-ron sâu.

2.5 decision trees cây quyết định (Chương 6): 12 2.5 Decision Trees (Cây quyết định) 13 2.6 Random Forests (Rừng ngẫu nhiên) 13 2.7 Gradient Boosting 13 3

Chapter 4 đánh giá trong trí tuệ nhân (Chương 6): Chapter 4 Đánh Giá Trong Trí Tuệ Nhân Tạo - Evaluation 4.1.2 Độ Chính Xác Của Dự Đoán Dương (Precision) Precision đo lường trong số các dự đoán dương thì có bao nhiêu là thực sự dương

Kế hoạch ôn tập (Review Plan)

Ôn tập nhanh 10 phút:

Ôn tập nhanh thuật ngữ và ý cốt lõi từng bài.

Ôn tập 30 phút:

Trả lời câu hỏi tự kiểm tra nhanh (Quick Check).

Ôn tập chuyên sâu 1 giờ:

Hệ thống hóa kiến thức và vận dụng vào bài tập tình huống thực tế.